Zadaća 5 iz osnova operacionih istraživanja

Krajnji rok za predaju zadaće je **nedjelja**, **10. XII 2024.** (do kraja dana).

Zadatak 1

Potrebno je u Julija programskom jeziku napraviti opću funkciju nazvanu `general_simplex` za rješavanje općih problema linearnog programiranja. Funkcija treba da ima sljedeću sintaksu:

```
Z,X,Xd,Y,Yd,status=general_simplex(goal,c,A,b,csigns,vsigns);
```

Ulazni parametri funkcije imaju sljedeće značenje:

`goal`	Ovaj parametar može imati samo vrijednost `'max'` ili `'min'`, ovisno da li tražimo minimum ili maksimum. Svaka druga vrijednost signalizira grešku.
`c`	Vektor (tj. matrica sa jednom kolonom) koja sadrži koeficijente funkcije cilja. Ovaj vektor ima onoliko elemenata koliko ima promjenljivih.
`A`	Matrica čiji su elementi koeficijenti sa lijeve strane ograničenja (tehnološka matrica). Ova matrica ima onoliko redova koliko ima ograničenja (ne računajući ograničenja na znak promjenljivih) a onoliko kolona koliko ima promjenljivih.
`b`	Vektor koji sadrži koeficijente sa desne strane ograničenja. Ovaj vektor ima onoliko elemenata koliko ima ograničenja (ne računajući ograničenja na znak promjenljivih). Funkcija treba da radi ispravno *bez obzira kakav je znak elemenata ovog vektora*.
`csigns`	Vektor čiji su elementi isključivo `+1`, `-1` ili `0` (svaka druga vrijednost signalizira grešku) i koji ima onoliko elemenata koliko ima ograničenja (ne računajući ograničenja na znak promjenljivih). Svaki element odgovara jednom ograničenju i on je `+1` ukoliko je odgovarajuće ograničenje tipa "veće ili jednako", `-1` ukoliko je odgovarajuće ograničenje tipa "manje ili jedanko" i `0` ukoliko je odgovarajuće ograničenje tipa jednakosti. Ukoliko se ovaj i naredni parametar izostave, podrazumijeva se da su sva ograničenja tipa "manje ili jednako" za problem maksimizacije, odnosno "veće ili jednako" za problem minimizacije.
`vsigns`	Vektor čiji su elementi isključivo `+1`, `-1` ili `0` (svaka druga vrijednost signalizira grešku) i koji ima onoliko elemenata koliko ima promjenljivih. Svaki element odgovara jednoj promjenljivoj i on je `+1` ukoliko odgovarajuća promjenljiva mora biti nenegativna, `-1` ukoliko odgovarajuća promjenljiva mora biti nepozitivna i `0` ukoliko odgovarajuća promjenljiva nije ograničena po znaku. Ukoliko se ovaj parametar izostavi, podrazumijeva se da sve promjenljive moraju biti nenegativne.

Izlazni parametri (rezultati) funkcije imaju sljedeće značenje:

`Z`	Optimalna vrijednost funkcije cilja.
`X`	Vektor koji sadrži optimalne vrijednosti izvornih promjenljivih (tj. promjenljivih koje su postojale u problemu prije dodavanja izravnavajućih promjenljivih).
`Xd`	Vektor koji sadrži optimalne vrijednosti izravnavajućih promjenljivih. Ovaj izlazni parametrar kao i svi iza njega mogu se izostaviti.
`Y`	Vektor koji sadrži cijene u sjeni za sve resurse, tj. optimalne vrijednosti dualnih promjenljivih. Voditi računa da njihov znak bude uvijek ispravan.
`Yd`	Vektor koji sadrži reducirane cijene, tj. optimalne vrijednosti izravnavajućih dualnih promjenljivih.

`status`	Vrijednost koja opisuje kako se postupak završio. Ovaj parametar može imati sljedeće vrijednosti: 0: Postupak je okončan normalno i nađeno je jedinstveno nedegenerirano optimalno rješenje. 1: Postupak je okončan normalno i nađeno je jedinstveno degenerirano optimalno rješenje. 2: Postupak je okončan normalno i nađeno je jedno optimalno rješenje, ali koje nije jedinstveno (odnosno, postoje i druga optimalna rješenja. 3: Funkcija cilja nije ograničena na dopustivoj oblasti, odnosno optimalno rješenje se nalazi u beskonačnosti. Izlazni parametar `z` tada treba imati vrijednost `inf`, a vrijednosti ostalih izlaznih parametara nisu bitne. 4: Dopustiva oblast ne postoji, odnosno problem nema rješenja. Izlazni parametar `z` tada treba imati vrijednost `nan`, a vrijednosti ostalih izlaznih parametara nisu bitne. 5: Greška u zadavanju parametara, odnosno zadani parametri nisu smisleni (npr. ulazne matrice nemaju saglasne formate, parametar `goal` nije `'min'` ili `'max'`, itd.). Izlazni parametar `z` tada treba imati vrijednost `nan`, a vrijednosti ostalih izlaznih parametara nisu bitne.

Cijela funkcija treba da bude u jednom .jl-fajlu ili .ipynb-fajlu. Ukoliko Vam trebaju pomoćne funkcije, implementirajte ih unutar istog fajla kao lokalne funkcije. Za implementaciju funkcije **ne smijete** koristiti ugrađene Julia funkcije za optimizaciju kao što su `GLPK, JuMP` i slične.

Potrebno je da predate odgovarajući fajl (kao datoteku u `.ipynb ili .jl` formatu). Prije predaje testirati funkciju na testnim primjerima koji će biti objavljeni na c2 i navesti koji od primjera ne rade i zašto. Ove primjere ne možete koristiti kao testne primjere koji se traže u zadatku 2.

Zadatak 2

Testirajte napisanu funkciju `general_simplex` iz prethodnog zadatka na više različitih primjera, koji su različite prirode. Na primjer, trebali biste imati primjere i maksimizacije i minimizacije, zatim različite vrste ograničenja, slučajeve u kojima promjenljive nisu ograničene po znaku, primjere u kojima nema rješenja ili u kojima optimalno rješenje nije ograničeno, itd. Razne primjere pogodne za testiranje možete naći u predavanjima, s obzirom da ti primjeri imaju rješenja (kao primjer problema u kojem promjenljive nisu ograničene po znaku možete uzeti dualni problem problema na kojem je na predavanjima demonstriran više-za-manje ili manje-za-više paradoks). Primjere problema u kojima je rješenje neograničeno ili u kojima nema rješenja konstruišite sami. Rezultate testiranja opišite u kratkom izvještaju koji treba da sadrži kako je glasio problem, šta ste zadali kao ulazne parametre i šta ste dobili kao izlaz iz funkcije.

Potrebno je da predate izvještaj sa rezultatima testiranja u `.pdf` formatu.